

Exerciții — unelte care prind ce ți-a scăpat

16 exerciții pe atelierul tău din `ms\` și pe clusterul tău · lecția 2

Terenul de lucru: folderul tău `C:\Projects\teste claude\ms`, cu `.claude\` și `CLAUDE.md` deja acolo, plus `kubectl` pe contextul clusterului tău și `gh` autentificat.

Ce se schimbă față de fișa 1: atunci construiai prima unealtă. Acum ai deja un atelier și o săptămână de lucru cu el în urmă. Fișa asta se uită la **ce a trecut prin atelier fără să fie prins**.

Cum se citește fișa: fiecare exercițiu are o sarcină exactă, un **gata când** pe care îl poți bifa cu ochii tăi în repo sau în cluster, și — unde se poate — o **poartă**: strici ceva cu mâna și verifici că unealta se supără. Regula din fișa 1 rămâne singura care contează: **o unealtă care nu poate da niciodată verdict negativ nu e o unealtă, e un afiș**.

Partea I — Podeaua atelierului: ce poate vedea agentul

Numără repo-urile de două ori

1

Pornește Claude în `ms\` și cere-i, exact așa:

```
Cate repo-uri git sunt in folderul asta? Pentru fiecare, scrie
numele folderului, remote-ul complet si data ultimului commit.
```

Apoi, în alt terminal, lista adevărată:

```
gh repo list nimigeanconstantinon --limit 20
```

Gata când: ai cele două liste una lângă alta și ai răspuns la două întrebări: care repo-uri apar la `gh` și **nu** apar în folder, și — pentru fiecare folder — **ultimul segment din remote e același cuvânt cu numele folderului?** Unde nu e, notează-l.

De ce începe fișa aici: un repo care nu e pe podea nu există pentru agent. O unealtă care compară două lucruri, dintre care unul lipsește, nu se plânge — raportează liniștit `OK` pe ce a găsit. Iar un folder cu numele unui repo și remote-ul altuia e mai rău decât unul lipsă: acolo unealta răspunde încrezătoare, din fișierul greșit.

Pune pe podea tot ce scrie sau citește platforma

2

Din lista de la exercițiul 1, clonează în `ms\` fiecare repo despre care nu poți spune, fără să te uiți, că nimeni nu mai scrie în el. Apoi repune cuvânt cu cuvânt întrebarea de la exercițiul 1.

Gata când: numărul din răspuns e egal cu numărul de repo-uri pe care le-ai decis tu că fac parte din platformă, iar pentru fiecare repo pe care l-ai lăsat afară poți spune în ce condiție ai afla că te-ai înșelat.

Numele folderului nu e numele repo-ului

Folderele tale nu se cheamă ca repo-urile de pe GitHub, iar în cluster lucrurile au al treilea nume. Cere-i un tabel cu patru coloane:

Fa un tabel: folderul local, repo-ul de pe GitHub (din remote), ce rulează în cluster din el, și namespace-ul. Ia numele din fișiere, nu din numele folderelor. Scrie pentru fiecare rând de unde ai luat informația.

Mută tabelul, ca atare, în `CLAUDE.md`, și pune pe prima coloană numele exact al folderului tău, cu tot cu diferențele de scriere.

Gata când: într-o sesiune nouă, la întrebarea In ce repo scrie CI-ul din importer? răspunsul citează un fișier de workflow și o linie din el — nu numele unui folder.

Coloana a doua e singura care contează. Ea se ia din `git remote -v`, nu din numele folderului. Un folder poate fi botezat cu numele unui repo și să conțină altul — și atunci tot ce urmează, de la `CLAUDE.md` până la ultima unealtă, e corect construit peste o premisă falsă.

CLAUDE.md-ul tău, citit de un străin

Ai scris harta din memorie. Pune-l să găsească ce lipsește din ea:

Citeste doar `CLAUDE.md`, nimic altceva. Spune-mi trei întrebări despre deploy la care NU poți răspunde din el, și de ce ti-ar trebui răspunsul ca să nu ghicești.

Gata când: ai trei întrebări, ai ales două și ai completat răspunsul lor în `CLAUDE.md`. A treia o lași deliberat afară și notezi de ce.

Partea II — Unelte care răspund la clasa de probleme, nu la cazul de azi

/lant v2 — nu doar valoarea, ci direcția

Prima versiune compara tag-uri. Un tag care coincide nu dovedește nimic dacă cele două valori nu circulă pe același drum. Cere versiunea a doua, cu direcția în ea:

Pentru fiecare serviciu care are pipeline (data-service, importer-service, client): scoate un tabel cu cinci coloane.

1. serviciul
2. fișierul de workflow și linia în care scrie în ce repo se face bump-ul de tag
3. Application-ul ArgoCD care îl deploiează și repoURL-ul din el
4. tag-ul aflat în fiecare din cele două locuri
5. verdict: același repo / repo diferit

Dacă un serviciu nu are Application, scrie asta pe rând propriu, nu-l sari. Citeste doar fișiere și git log, nu modifica nimic.

Gata când: ai un rând pentru fiecare serviciu și, pe coloanele 2 și 3, o cale de fișier cu număr de linie. Fără căi, tabelul nu e verificabil și exercițiul nu e terminat.

Ce schimbă coloana 5: un tabel de valori îți spune dacă azi e bine. Un tabel de drumuri îți spune dacă mâine poate fi bine fără să pui tu mâna.

Poarta pentru /lant

1. schimbă cu mâna o cifră dintr-un tag, în repo-ul din care citește ArgoCD — fără să comiți;
2. rulează /lant ;
3. pune cifra la loc și verifică cu `git status` și `git diff`.

Gata când: la pasul 2 a ieșit verdict negativ pe serviciul potrivit, cu ambele valori afișate, iar la pasul 3 `git diff` e gol în toate repo-urile.

Aceeași întrebare, pusă la plural

Ai trei fișiere de pipeline scrise în momente diferite. Cere comparația, nu o inspecție:

Compara cele trei fișiere de workflow linie cu linie. Scoate un tabel cu fiecare diferența reală dintre ele și, pentru fiecare, spune dacă e explicată de faptul că sunt servicii diferite (limbaj, pași de build) sau e o ramă dintr-o versiune mai veche.

Gata când: ai găsit cel puțin o diferență pe care nu o explică „sunt servicii diferite” — și ai deschis fișierul ca să-l vezi cu ochii tăi.

Tiparul pe care îl atacă exercițiul: ceri fix-ul pe fișierul la care te uiți, agentul îl face impecabil, și nimeni nu întreabă „și celelalte două?”. Întrebarea aia rămâne a ta — agentul răspunde la ce ai pus, nu la ce ai vrut.

De unde vine valoarea — regula, nu linia

Alege o valoare pe care ai corectat-o tu cu mâna, recent, într-un `values.yaml`. Întrebarea nu e „e corectă acum”, ci „cine o scrie de obicei”:

Cine scrie în mod normal valoarea asta? Arată-mi linia exactă din pipeline care o rescrie, și spune ce formă va avea valoarea după următoarea rulare.

Proba, obligatorie: copiază fișierul într-un folder temporar, rulează acolo, cu mâna, exact comanda din pipeline, cu o valoare de test aleasă de tine, și uită-te la rezultat.

Gata când: poți spune, arătând linia, dacă următoarea rulare păstrează corectura ta sau o șterge. Dacă o șterge, ai reparat o linie, nu o regulă — și știi asta din probă, nu din opinia agentului.

Skill /orfane — fișiere pe care nu le mai citește nimeni

Un fix care lasă în urmă versiunea veche a fișierului e o capcană pentru următorul om care deschide repo-ul — de obicei tu, peste trei luni. Construiește un skill:

Fa un skill /orfane care caută, în toate repo-urile din ms: fișiere de configurare (*.conf, *.yaml de chart, Dockerfile*) care nu sunt referite de niciun Dockerfile, workflow, chart sau manifest. Pentru fiecare: calea fișierului, și unde ai cautat referințe ca să conchizi că nu există.

Poarta: creează `dovada-orfan.conf` cu o linie oarecare, rulează skill-ul, apoi șterge fișierul.

Gata când: ți-a găsit fișierul plantat, ai citit lista întreagă, și pentru fiecare orfan raportat ai confirmat singur cu o căutare după numele fișierului că nu apare nicăieri.

Partea III — Al doilea rol: subagentul care n-are voie să fie de acord

Subagentul recenzent

10

Cine a scris o schimbare e ultimul care-i vede gaura. Cere un subagent cu un singur rol:

```
Fa un subagent "recenzent-schimbare". Primește un hash de commit sau un numar de PR. Citeste DOAR diff-ul si mesajul. Intoarce trei sectiuni: (a) ce spune mesajul ca face, (b) ce face diff-ul de fapt, (c) ce afirma mesajul si nu se poate verifica din diff-ul asta – pentru ca depinde de alt repo, de alta imagine sau de o alta schimbare. Nu propune fix-uri si nu da note.
```

Rulează-l pe ultimele trei commit-uri pe care le-ai făcut într-o singură seară.

Gata când: la cel puțin unul, secțiunea (c) nu e goală — și ai verificat tu, în celălalt repo, dacă afirmația se susține sau nu.

Ordinea, nu conținutul

11

Trei schimbări corecte, făcute în ordinea greșită, lasă o fereastră în care platforma nu e nici în starea veche, nici în cea nouă. Cere reconstituirea:

```
Din git log-ul tuturor repo-urilor din ms, reconstituie ordinea in timp a schimbarilor dintr-o singura seara, cu ora fiecareia. Pentru fiecare pereche de schimbari care depind una de alta, spune care trebuia sa fie prima si cat a durat fereastra dintre ele.
```

Gata când: ai lista cronologică cu ore și cel puțin o pereche unde ordinea reală diferă de ordinea sigură. Scrie fereastra în minute.

De ce subagent și nu aceeași conversație

12

Cere-i, în sesiunea obișnuită, să propună o schimbare mică într-un fișier de configurare — și imediat după, în aceeași conversație, să o critice dur. Apoi dă aceeași schimbare subagentului de la exercițiul 10.

Gata când: ai ambele verdicte în față și poți spune care e mai dur. Țasta e tot argumentul pentru subagenți: nu sunt mai deștepți, sunt doar **neinvestiți**.

Partea IV — Ce se întâmplă fără să ceri

Hook-ul cu trei verificări

13

Pune un hook înainte de commit care oprește trei lucruri:

- orice fișier care conține markeri de conflict (<<<<<<<);
- orice valoare sensibilă scrisă în clar;
- orice fișier care nu e al tău — documente de review, soluții, note primite de la altcineva, ajunse în repo din greșeală.

Gata când: ai testat **fiecare** din cele trei separat, provocându-l cu mâna, și de fiecare dată commit-ul s-a oprit cu motivul pe ecran. Trei teste, nu unul.

/secrete v2 — toate aparițiile, nu prima

14

O valoare sensibilă găsită într-un fișier și rezolvată acolo rămâne, de regulă, în încă două locuri: un script și un fișier de configurare copiat. Cere gruparea pe valoare:

```
Extinde skill-ul /secrete: pentru fiecare valoare sensibila
gasita, cauta exact aceeasi valoare in toate repo-urile si listeaza
TOATE fisierele si liniile in care apare. Grupeaza rezultatul pe
valoare, nu pe fisier. Referintele de tip secretKeyRef si continutul
criptat nu se raporteaza.
```

Poarta: pune aceeași linie inventată în două fișiere din două repo-uri diferite, rulează, apoi scoate-le.

Gata când: a raportat **o singură intrare cu două fișiere**, nu două intrări separate. Diferența dintre cele două forme e diferența dintre a rota o valoare și a crede că ai rotit-o.

Ce e în repo-urile tale și nu ai scris tu

15

```
Listeaza fisierele din repo-urile din ms care nu au fost scrise
de mine. Verifica in git log cine le-a adus si cand. Grupeaza pe repo si
spune pentru fiecare daca mai e folosit de ceva din proiect.
```

Gata când: ai lista completă, ai decis pentru fiecare fișier *rămâne* sau *se șterge*, și ai făcut curățenia cu un commit al tău. Repo-urile tale sunt publice: ce e acolo e citit ca fiind al tău.

Jurnalul de o zi

16

La finalul unei zile în care ai lucrat, nu la finalul fișei:

```
Din git log-ul tuturor repo-urilor din ms, spune-mi ce am atins
azi, in ce ordine, si ce a ramas neterminat: fisiere modificate fara
commit, branch-uri nemergeate, pipeline-uri care n-au rulat, valori
schimbate intr-un repo si nu in celalalt.
```

Gata când: în listă apare cel puțin un lucru pe care îl uitaseși. Alea, adunate, sunt lucrurile care peste o lună se cheamă „nu mai știu de ce nu merge”.

Ce au în comun toate cele 16: niciuna dintre uneltele de aici nu e mai deșteaptă decât tine. Fac trei lucruri pe care tu le faci doar când ești odihnit și ai timp: se uită la **toate** cazurile, nu la cel de azi; spun **de unde** au aflat fiecare valoare; și pot da verdict negativ. Restul — ce întrebi, și dacă crezi răspunsul — rămâne la tine, și rămâne partea grea.